講義 19：除錯 Debugger（授課順序：接在講義 5 之後）

> 對應練習：ex19（附一支埋了 3 個 bug 的程式）｜答案程式：`ZR_TR19_DEBUGGING`（修正版）

本講重點

- 三種進入 Debugger 的方式：`/h`、Session 中斷點、`BREAK-POINT`
- 單步鍵 F5 / F6 / F7 / F8 的分工
- 看變數值、看 internal table、改變數值重跑
- Watchpoint：值變了才停
- ST22 dump 分析：從當掉紀錄找回「案發那一行」
- 除錯的思考方法：先假設、再驗證

1. 進入 Debugger 的三種方式

方式	操作	適用場景
命令欄 `/h`	執行前在命令欄輸入 `/h`（狀態列顯示 Debugging switched on）再執行	想從頭跟一遍
Session 中斷點	SE38 編輯器中游標停在某行 → 工具列「Set/Delete Session Breakpoint」（或 Ctrl+Shift+F12）	**最常用**：直接停在懷疑的那一行；只對自己這次登入有效，登出即消失
`BREAK-POINT` 陳述式	在程式裡寫死 `BREAK-POINT.` 或 `BREAK 帳號.`	臨時手段。**團隊規範：commit / 傳輸前必須清乾淨**（`BREAK 帳號.` 只攔指定帳號，忘了刪也很失禮）

進入後的畫面（New Debugger）：上方是原始碼與目前停的行（黃色箭頭）、下方是變數區；桌面（Desktop）分頁可切換不同工具佈局。

2. 單步四鍵（背起來）

鍵	名稱	效果
F5	Step Into	走一步；遇到 PERFORM / CALL FUNCTION **走進去**
F6	Step Over	走一步；遇到 PERFORM / CALL FUNCTION **整個跑完跳過**（不進去）
F7	Return	把目前這層（FORM/FM）跑完，**回到呼叫端**
F8	Continue	一路跑到下一個中斷點（沒有就跑完程式）

實戰節奏：F8 衝到懷疑區域附近的中斷點 → F6 逐行走主流程 → 確認要進哪個 FORM 再 F5 進去 → 進錯了 F7 出來。

3. 看變數、看內表、改值

- **看單值**：原始碼裡**雙擊變數名**，變數與目前值出現在下方變數區；結構會顯示欄位展開鈕。
- **看 internal table**：雙擊內表名 → 變數區點它 → 切到 Table 檢視，整張表的內容逐列顯示（也能看目前 `sy-tabix` 停在哪筆）。
- **系統欄位（System Field）**：變數區直接輸入 `sy-subrc`、`sy-tabix` 觀察——READ TABLE 之後 `sy-subrc` 是多少，眼見為憑。
- **改值**：變數區點進值的欄位改掉按 Enter（鉛筆模式）。用途：不改程式先驗證「如果這裡是 0 會怎樣」，或跳過還沒修的前置錯誤繼續往下查。改值只影響**這一次執行**。

4. Watchpoint：值變了才停

「`gv_total` 不知道被誰改壞了」——逐行追太慢，改設 Watchpoint：**該變數一被改動（或條件成立），程式就立刻停下**，停的那一行就是改掉它的程式碼。要抓「全域變數被莫名改掉」，這招最快。

4.1 建立

1. 先用中斷點或 `/h` 進入 Debugger（Watchpoint 只能在 Debugger **裡面**建立，SE38 編輯器裡沒有這個功能）。
2. 工具列按 **Watchpoint**（或選單 **Breakpoints → Create Watchpoint**），跳出「Create Watchpoint」對話框。
3. **Variable** 填變數名（如 `gv_total`）。區域變數（FORM 裡的 `DATA`）要等程式**停在那個 FORM 裡面**時再建，不然 Debugger 找不到這個變數。
4. 需要的話在 **Free Condition Entry** 填條件（如 `gv_total > 1000`）：填了就「條件成立才停」，不填就「值一變就停」。
5. 按 Enter 確認後按 F8 繼續執行，等它停下。

停下時，狀態列會出現「Watchpoint reached」之類的訊息，黃色箭頭停在**剛改完變數的下一行**，所以要往上看一行才是改掉它的陳述式。

4.2 查看已建立的 Watchpoint

切到 Debugger 上方的 **Break./Watchpoints** 分頁 → 再點其中的 **Watchpoints** 子分頁。清單列出這次除錯建立的所有 Watchpoint，包括變數名、所屬程式與條件。建了好幾個、忘了設過什麼的時候，到這裡查。

4.3 修改、停用、刪除

在上述清單中選一列，用清單上方的按鈕操作：

動作	用途
Change（鉛筆圖示）	重新打開對話框，改條件（例如把 `> 1000` 改成 `> 5000`）或改變數
Activate / Deactivate	暫時停用、不刪除；停用後不會觸發，之後可以再啟用
Delete（垃圾桶圖示）	刪除這個 Watchpoint

常用做法：**條件設太寬（每圈迴圈都停）→ Change 把條件收窄**，不用刪掉重建。

4.4 注意事項

- Watchpoint **只在這一次除錯期間有效**，Debugger 關掉就消失，不會像 Session 中斷點那樣留到登出，下次要再建。
- 監看的變數一多，每執行一行 Debugger 都要比對值，執行會變慢，用完就刪。
- 內表（internal table）也能設 Watchpoint，表的內容一有增刪改就會停，適合查「這張表是在哪一行被清空的」。

> SAP GUI 各版本的按鈕位置與名稱可能稍有不同；如果你看到的畫面跟這裡描述的不一樣，請回報，講義會再修正。

5. ST22：程式已經當掉了怎麼查

程式 dump（執行期錯誤（Runtime Error））時不要只截圖錯誤畫面——進 **ST22** 找這筆紀錄，重點看四個段落：

段落	內容
Category / Runtime Errors	錯誤類型（如 COMPUTE_INT_ZERODIVIDE）
Error analysis	白話說明哪裡不對
Source Code Extract	**案發那一行**（前後原始碼，>>>>> 標記）
Chosen variables	當下相關變數的值

常見 dump 類型對照（前面講次都預告過）：

Runtime Error	成因	講次
COMPUTE_INT_ZERODIVIDE	除以零	17
CONVT_NO_NUMBER	字元轉數值但內容不是數字	2
GETWA_NOT_ASSIGNED	field-symbol 未指派就使用	16
TIME_OUT	迴圈沒出口或 SELECT 太大	17 / 6
DBIF_RSQL_INVALID_RSQL	Open SQL 語法/條件組出問題	6

6. 除錯的思考方法

工具只是手段，方法才省時間：

1. **先重現**：找到穩定重現的輸入條件（哪個帳號、哪組選擇畫面（Selection Screen）值）。不能重現的 bug 先想辦法重現。
2. **提出假設**：根據症狀猜「資料錯」還是「邏輯錯」——輸出是殘留值？多半是 sy-subrc 沒檢查（講義 4）。改了沒生效？多半是忘了 MODIFY / 忘了啟用（講義 5 / 1）。
3. **用 Debugger 驗證假設**：中斷點下在假設的案發點，看變數值對不對——而不是漫無目的逐行走。
4. **修一個、驗一個**：一次修一個 bug 就重跑驗證，不要憑感覺一口氣改三處。

7. 常見錯誤與陷阱

症狀	原因
/h 打了沒反應	要在**執行前**輸入；打完那次執行才生效
中斷點隔天不見了	Session 中斷點登出即失效——是特性不是 bug（要長存用 External Breakpoint，進階）
F5 進到看不懂的標準程式深處	走進 SAP 標準呼叫了——F7 逐層返回，下次該處用 F6
正式機想 debug 改資料	生產環境的除錯/改值權限受控，本來就不該有——在 DEV/QAS 重現
BREAK-POINT 留到傳輸	檢查清單有這條（transport-flow 規範）：release 前全文搜尋 BREAK

8. 課堂練習

完成 ex19：題目附一支「會動但全是錯」的程式（3 個 bug：殘留值、白改、除零 dump），**規則是先用 Debugger 定位證據，再動手修**——每個 bug 要能說出「停在哪一行、看到什麼值、為什麼錯」。